SYSTEM PROJECT: Student Record Management System (documentation)

ER Diagram

	students	
PK	id	INT AUTO_INCREMENT
UQ	student_number	VARCHAR(20) NOT NULL
	first_name	VARCHAR(100) NOT NULL
	last_name	VARCHAR(100) NOT NULL
	course	VARCHAR(100) NOT NULL
	year_level	INT NOT NULL
	created_at	TIMESTAMP DEFAULT CURRENT_TIMESTAMP

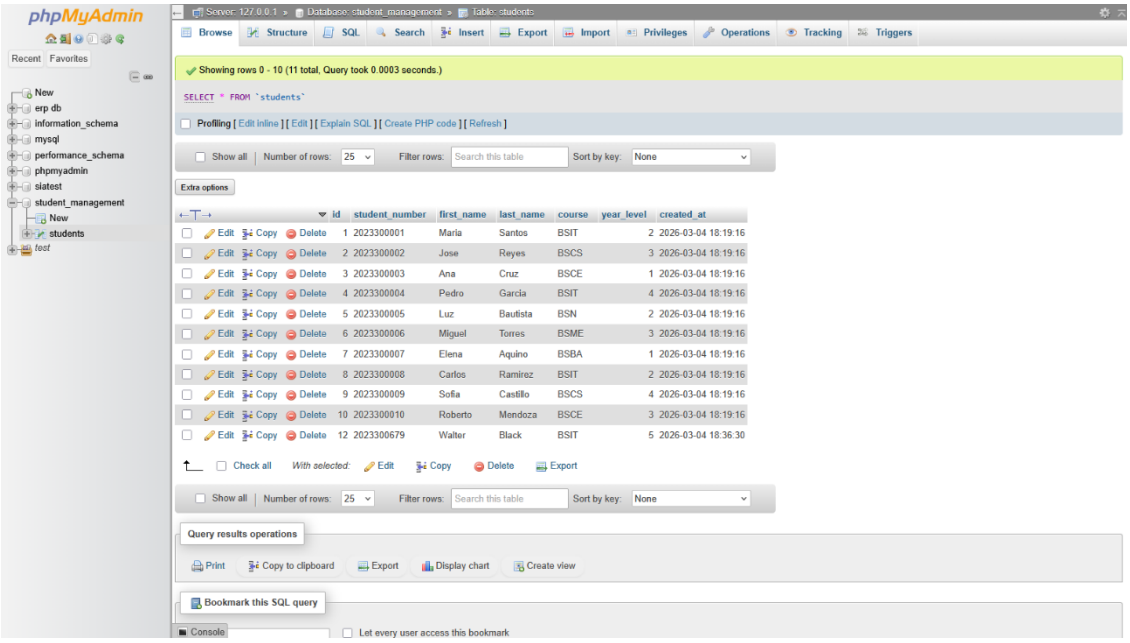
Legend:
PK = Primary Key (Unique identifier for each record)
UQ = Unique Constraint (No duplicates allowed)

API Documentation

Method	Endpoint	Description	Status Codes
POST	/api/students	Add new student	201, 400, 500
GET	/api/students	Get all students	200, 500
GET	/api/students/:id	Get single student	200, 400, 404, 500
GET	/api/students/search/:key word	Search by name/ID	200, 500
PUT	/api/students/:id	Update student	200, 400, 404, 500
DELETE	/api/students/:id	Delete student	200, 400, 404, 500

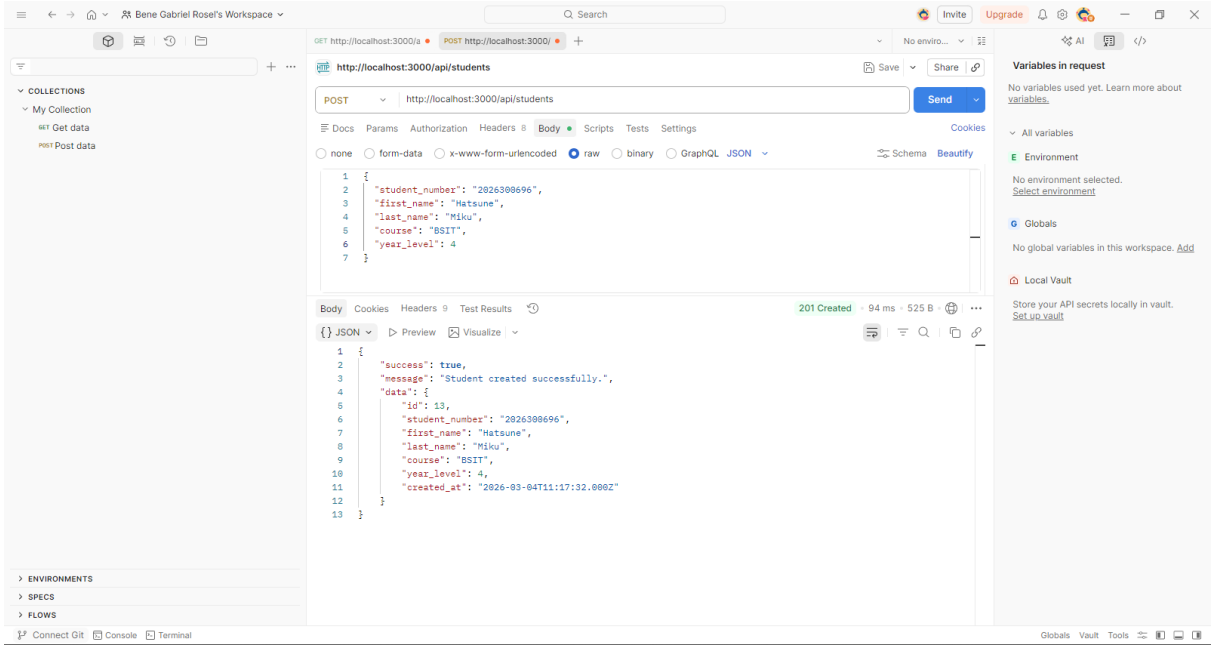
Screenshots

- Database Table

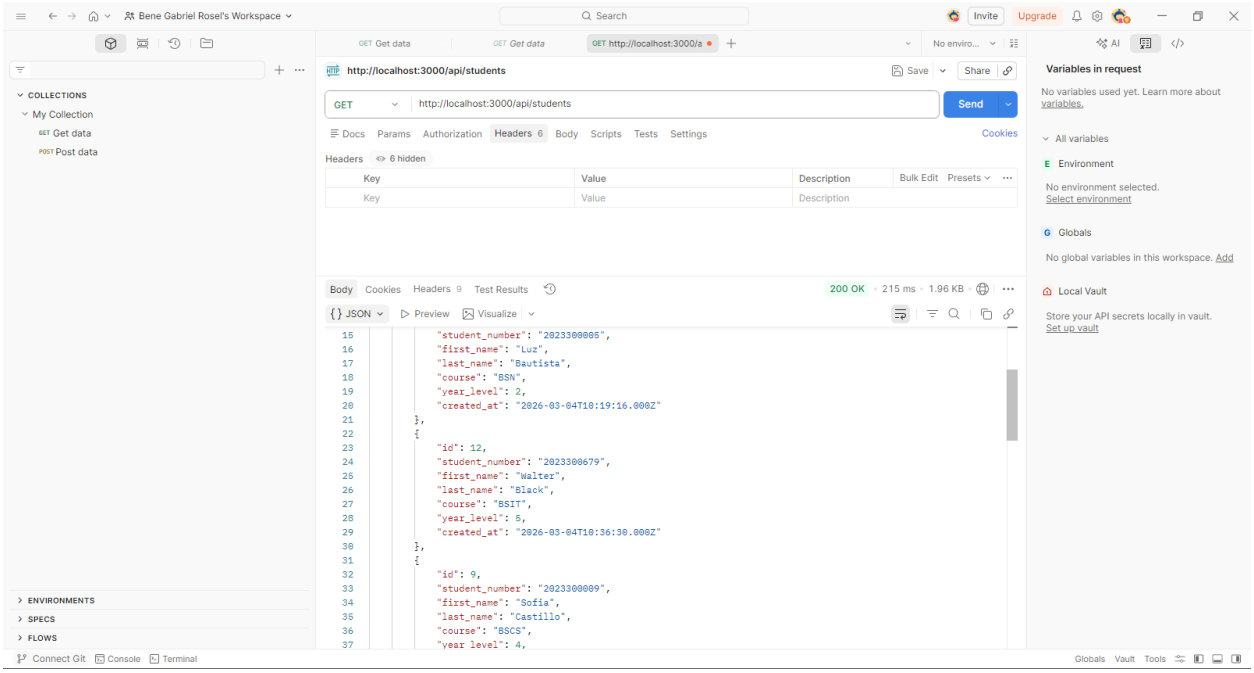


- Postman testing

I. Insert Student (POST)

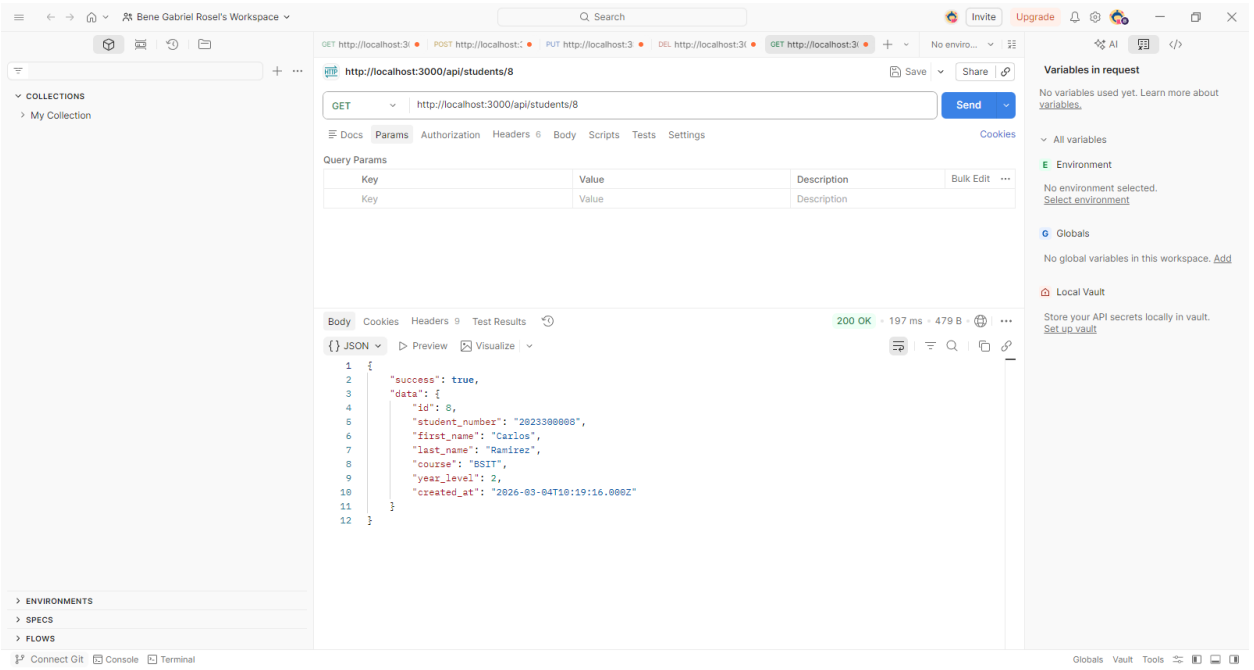


II. Select all students (GET)

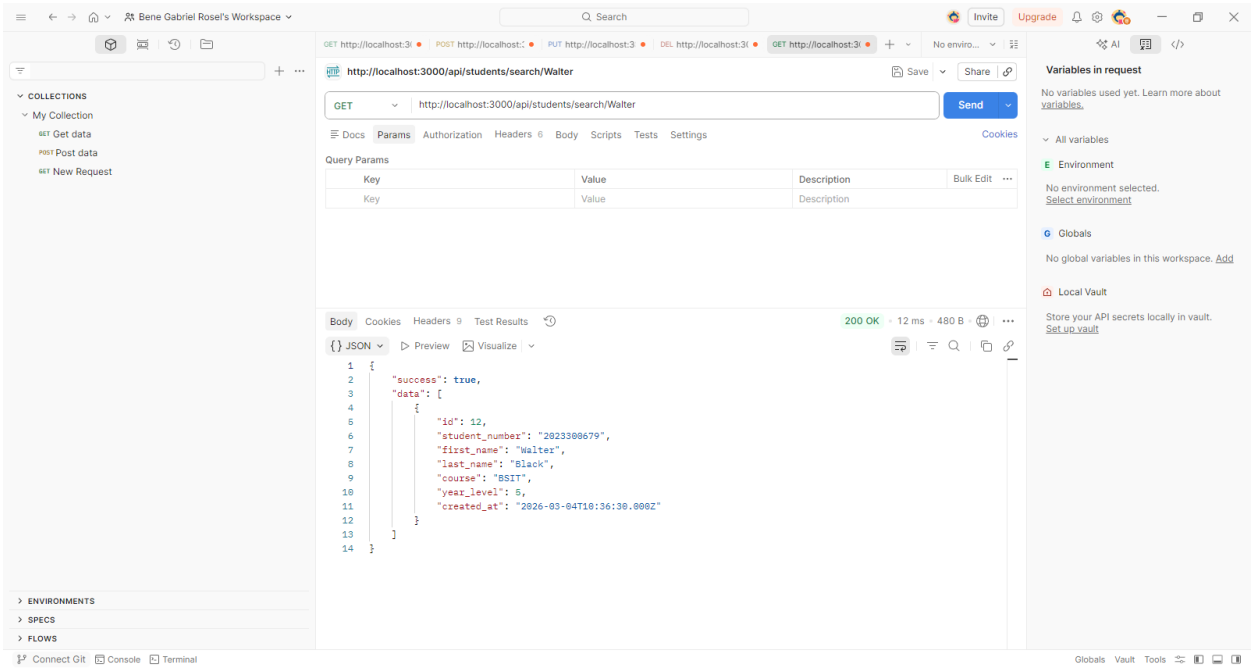


III. Select student (GET)

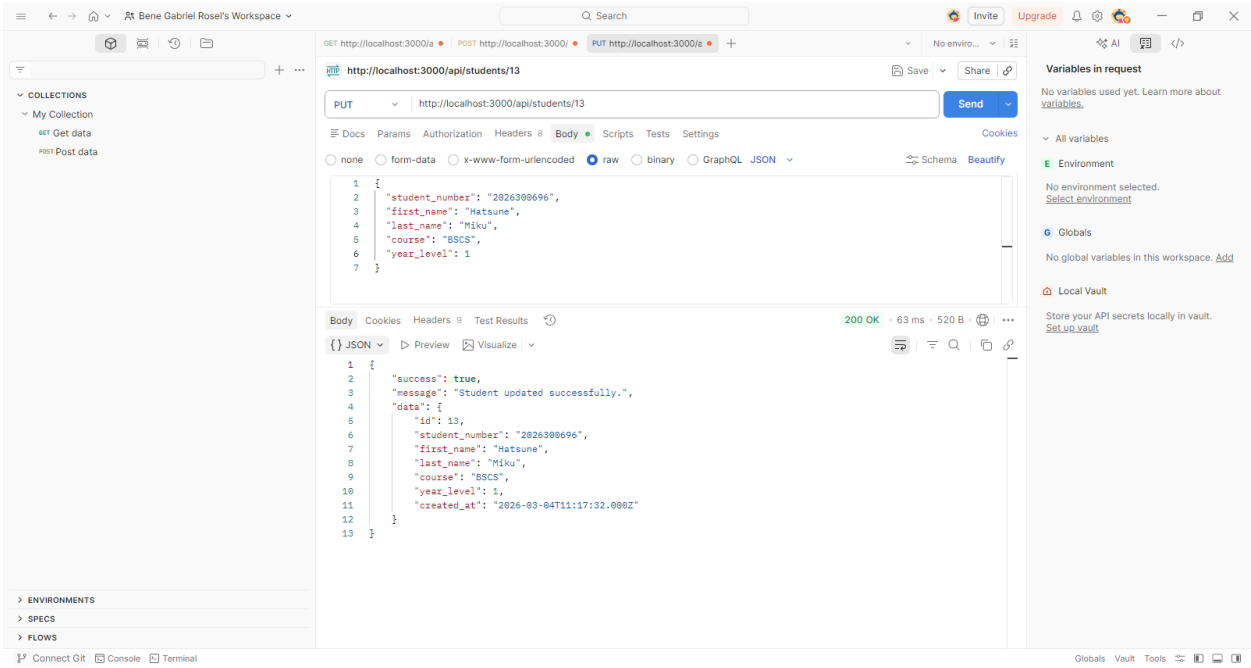
- By ID



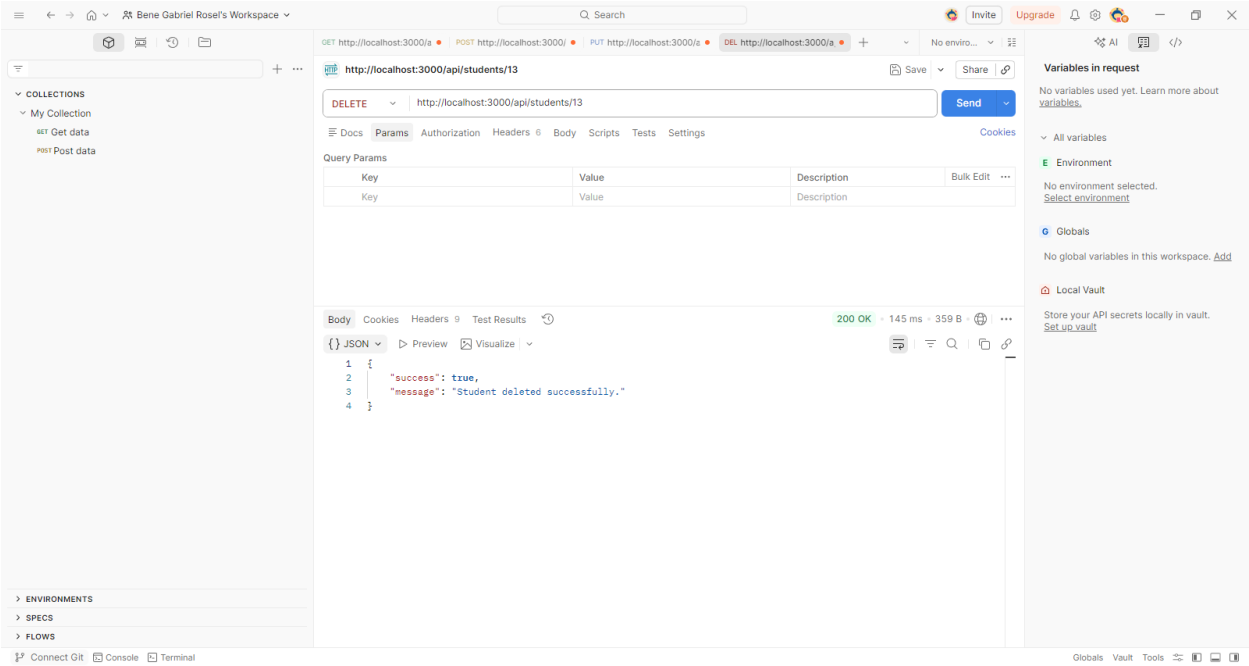
- By name/ID



IV. Update student (PUT)

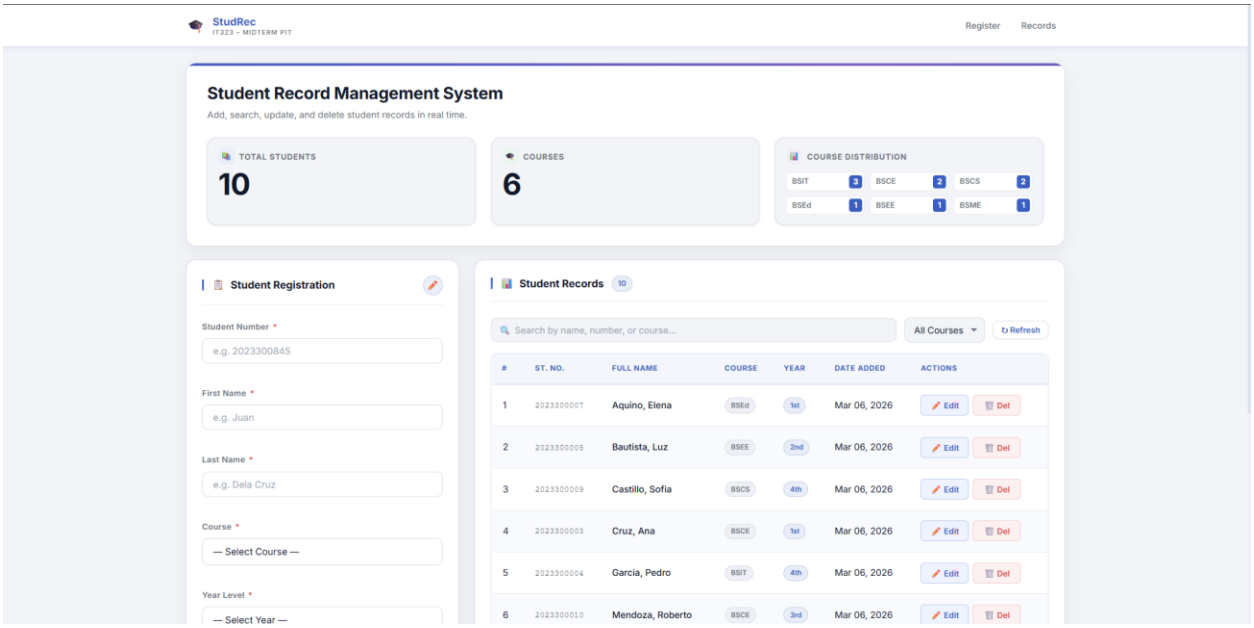


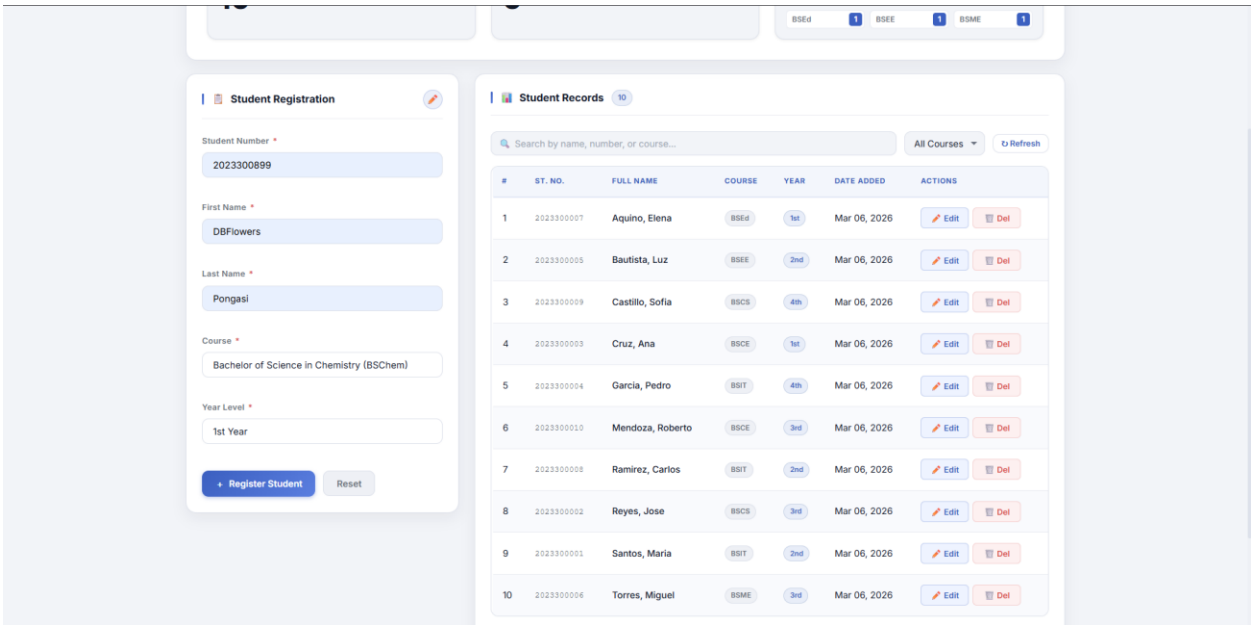
V. Delete student (DELETE)

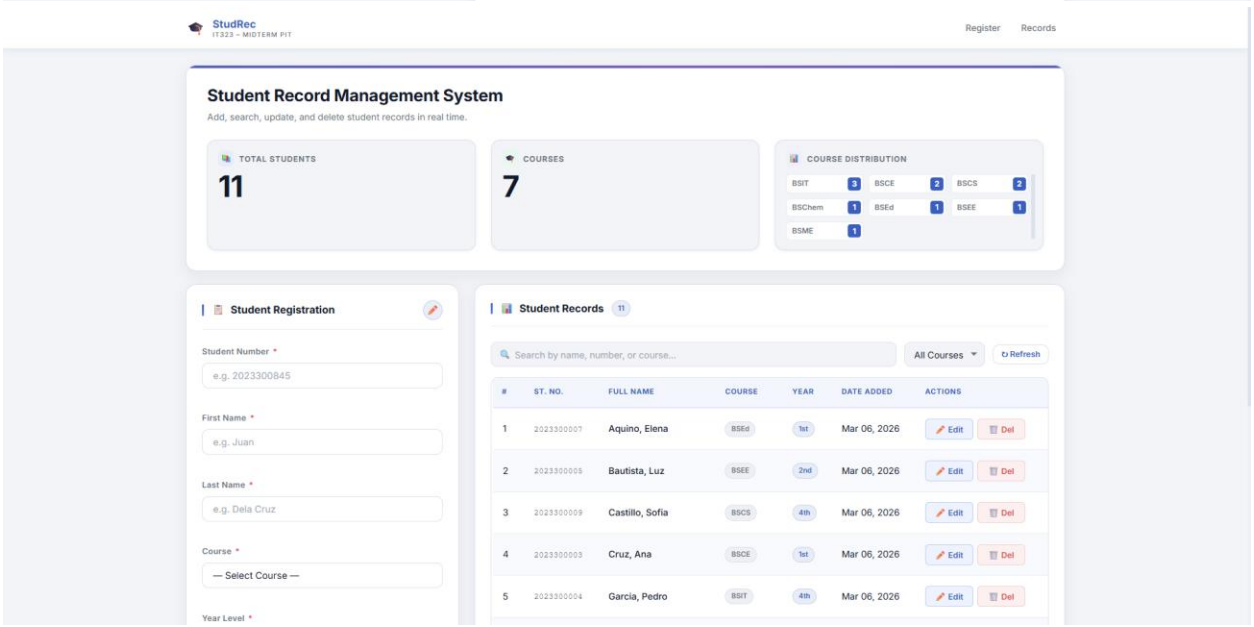


- Responsive Desktop View

I. Main Pages




BSEd 1
BSEE 1
BSME 1
Student Registration
Student Number *
2023300899
First Name *
DBFlowers
Last Name *
Pongasi
Course *
Bachelor of Science in Chemistry (BSChem)
Year Level *
1st Year
+ Register Student
Reset
Student Records 10
Search by name, number, or course...
All Courses
Refresh
ST. NO. FULL NAME COURSE YEAR DATE ADDED ACTIONS
1 2023300007 Aquino, Elena BSEd 1st Mar 06, 2026 Edit Del
2 2023300005 Bautista, Luz BSEE 2nd Mar 06, 2026 Edit Del
3 2023300009 Castillo, Sofia BSCS 4th Mar 06, 2026 Edit Del
4 2023300003 Cruz, Ana BSCE 1st Mar 06, 2026 Edit Del
5 2023300004 Garcia, Pedro BSIT 4th Mar 06, 2026 Edit Del
6 2023300010 Mendoza, Roberto BSCE 3rd Mar 06, 2026 Edit Del
7 2023300008 Ramirez, Carlos BSIT 2nd Mar 06, 2026 Edit Del
8 2023300002 Reyes, Jose BSCS 3rd Mar 06, 2026 Edit Del
9 2023300001 Santos, Maria BSIT 2nd Mar 06, 2026 Edit Del
10 2023300006 Torres, Miguel BSME 3rd Mar 06, 2026 Edit Del


StudRec
IT323 – MIDTERM PIT
Register
Records
Student Record Management System
Add, search, update, and delete student records in real time.
TOTAL STUDENTS
11
COURSES
7
COURSE DISTRIBUTION
BSIT 3
BSCE 2
BSCS 2
BSChem 1
BSEd 1
BSEE 1
BSME 1
Student Registration
Student Number *
e.g. 2023300845
First Name *
e.g. Juan
Last Name *
e.g. Dela Cruz
Course *
— Select Course —
Year Level *
Student Records 11
Search by name, number, or course...
All Courses
Refresh
ST. NO. FULL NAME COURSE YEAR DATE ADDED ACTIONS
1 2023300007 Aquino, Elena BSEd 1st Mar 06, 2026 Edit Del
2 2023300005 Bautista, Luz BSEE 2nd Mar 06, 2026 Edit Del
3 2023300009 Castillo, Sofia BSCS 4th Mar 06, 2026 Edit Del
4 2023300003 Cruz, Ana BSCE 1st Mar 06, 2026 Edit Del
5 2023300004 Garcia, Pedro BSIT 4th Mar 06, 2026 Edit Del

Student Number *

e.g. 2023300845

First Name *

e.g. Juan

Last Name *

e.g. Dela Cruz

Course *

— Select Course —

Year Level *

— Select Year —

+ Register Student | Reset

Search by name, number, or course... | All Courses | Refresh

#	ST. NO.	FULL NAME	COURSE	YEAR	DATE ADDED	ACTIONS
1	2023300007	Aquino, Elena	BSEd	1st	Mar 06, 2026	Edit Del
2	2023300005	Bautista, Luz	BSEE	2nd	Mar 06, 2026	Edit Del
3	2023300009	Castillo, Sofia	BSCS	4th	Mar 06, 2026	Edit Del
4	2023300003	Cruz, Ana	BSCE	1st	Mar 06, 2026	Edit Del
5	2023300004	Garcia, Pedro	BSIT	4th	Mar 06, 2026	Edit Del
6	2023300010	Mendoza, Roberto	BSCE	3rd	Mar 06, 2026	Edit Del
7	2023300899	Pongasi, DBFlowers	BSChem	1st	Mar 13, 2026	Edit Del
8	2023300008	Ramirez, Carlos	BSIT	2nd	Mar 06, 2026	Edit Del
9	2023300002	Reyes, Jose	BSCS	3rd	Mar 06, 2026	Edit Del
10	2023300001	Santos, Maria	BSIT	2nd	Mar 06, 2026	Edit Del

Showing 1-10 of 11 students

Prev 1 2 Next

© 2026 Student Record Management System - IT323 - Midterm PIT
Developed by Bene Gabriel Rosel - Joeremy Christie Ong - Vince Roy Lim

Built with Node.js · Express · MySQL · Vanilla JS

StudRec
IT323 – MIDTERM PIT

Register | Records

Student Record Management System

Add, search, update, and delete student records in real time.

TOTAL STUDENTS
11

COURSES
7

COURSE DISTRIBUTION

BSIT	3	BSCE	2	BSCS	2
BSChem	1	BSEd	1	BSEE	1
BSME	1				

Student Registration

Student Number *

e.g. 2023300845

First Name *

e.g. Juan

Last Name *

e.g. Dela Cruz

Course *

— Select Course —

Year Level *

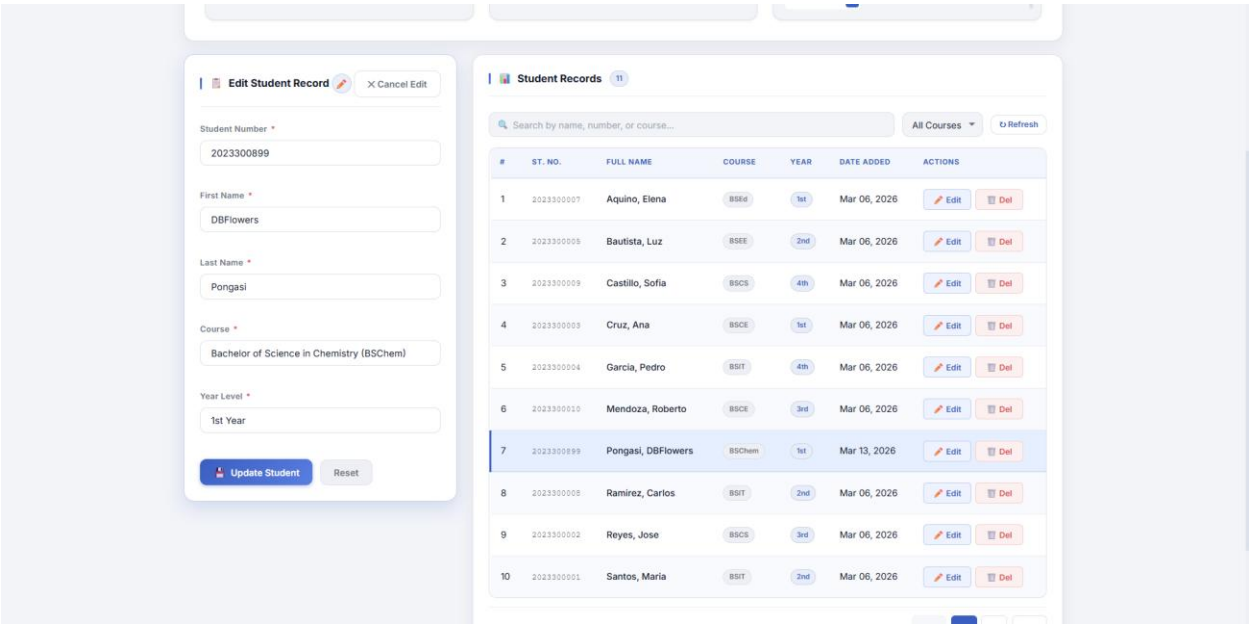
Student Records 11

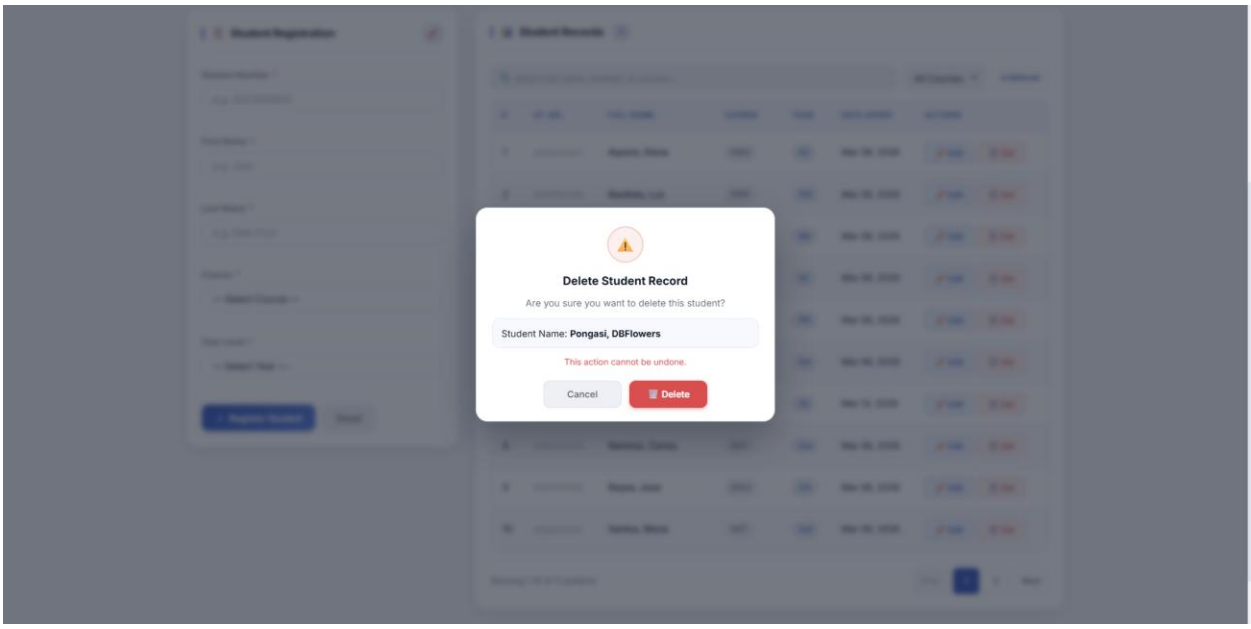
Po | All Courses | Refresh

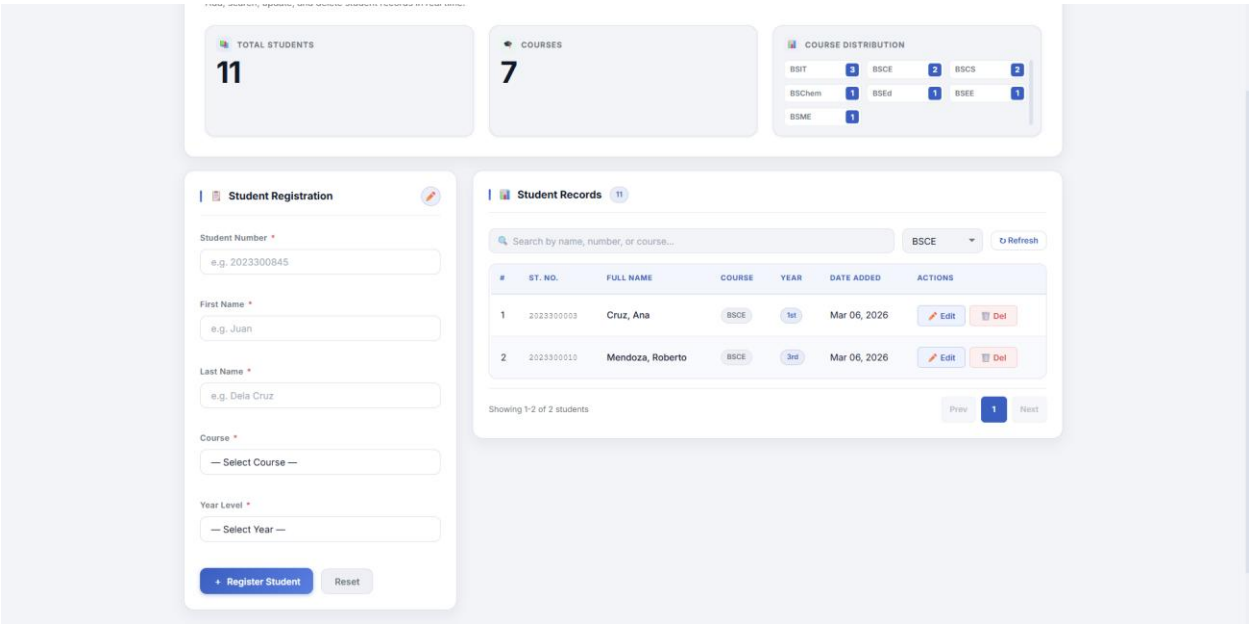
#	ST. NO.	FULL NAME	COURSE	YEAR	DATE ADDED	ACTIONS
1	2023300899	Pongasi, DBFlowers	BSChem	1st	Mar 13, 2026	Edit Del

Showing 1-1 of 1 students

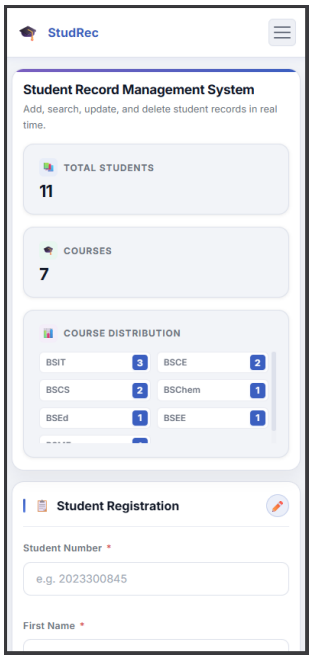
Prev 1 Next

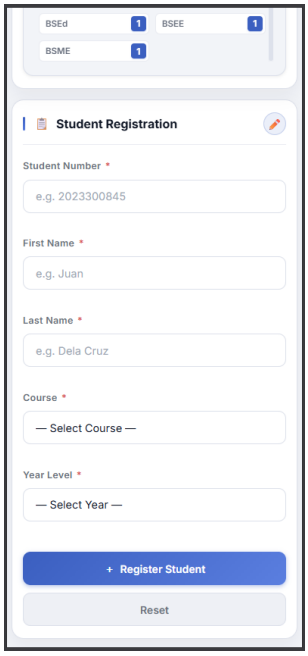

Edit Student Record
Cancel Edit
Student Number *
2023300899
First Name *
DBFlowers
Last Name *
Pongasi
Course *
Bachelor of Science in Chemistry (BSChem)
Year Level *
1st Year
Update Student
Reset
Student Records 11
Search by name, number, or course...
All Courses
Refresh
ST. NO. FULL NAME COURSE YEAR DATE ADDED ACTIONS
1 2023300007 Aquino, Elena BSEd 1st Mar 06, 2026 Edit Del
2 2023300005 Bautista, Luz BSEE 2nd Mar 06, 2026 Edit Del
3 2023300009 Castillo, Sofia BSCS 4th Mar 06, 2026 Edit Del
4 2023300003 Cruz, Ana BSCE 1st Mar 06, 2026 Edit Del
5 2023300004 Garcia, Pedro BSIT 4th Mar 06, 2026 Edit Del
6 2023300010 Mendoza, Roberto BSCE 3rd Mar 06, 2026 Edit Del
7 2023300899 Pongasi, DBFlowers BSChem 1st Mar 13, 2026 Edit Del
8 2023300008 Ramirez, Carlos BSIT 2nd Mar 06, 2026 Edit Del
9 2023300002 Reyes, Jose BSCS 3rd Mar 06, 2026 Edit Del
10 2023300001 Santos, Maria BSIT 2nd Mar 06, 2026 Edit Del

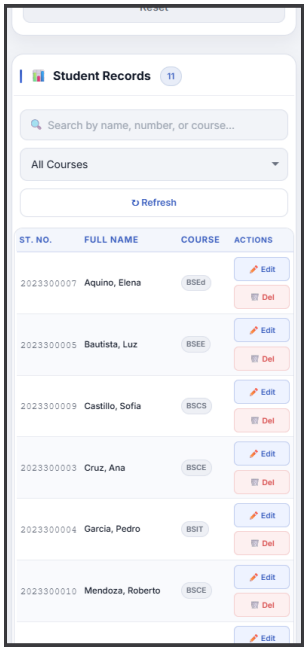

Delete Student Record
Are you sure you want to delete this student?
Student Name: Pongasi, DBFlowers
This action cannot be undone.
Cancel
Delete

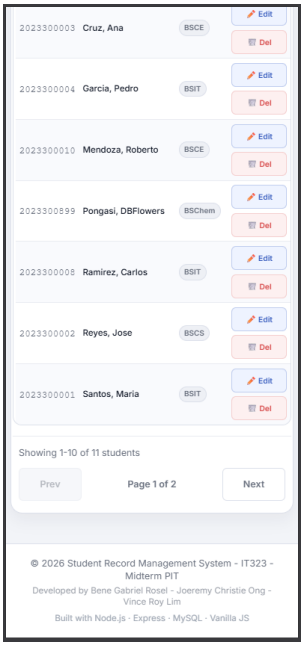

TOTAL STUDENTS
11
COURSES
7
COURSE DISTRIBUTION
BSIT 3
BSCE 2
BSCS 2
BSChem 1
BSEd 1
BSEE 1
BSME 1
Student Registration
Student Number *
e.g. 2023300845
First Name *
e.g. Juan
Last Name *
e.g. Dela Cruz
Course *
— Select Course —
Year Level *
— Select Year —
+ Register Student
Reset
Student Records 11
Search by name, number, or course...
BSCE
Refresh
ST. NO. FULL NAME COURSE YEAR DATE ADDED ACTIONS
1 2023300003 Cruz, Ana BSCE 1st Mar 06, 2026 Edit Del
2 2023300010 Mendoza, Roberto BSCE 3rd Mar 06, 2026 Edit Del
Showing 1-2 of 2 students
Prev 1 Next

- Mobile View


StudRec
Student Record Management System
Add, search, update, and delete student records in real time.
TOTAL STUDENTS
11
COURSES
7
COURSE DISTRIBUTION
BSIT 3
BSCE 2
BSCS 2
BSChem 1
BSEd 1
BSEE 1
Student Registration
Student Number *
e.g. 2023300845
First Name *


BSEd 1
BSEE 1
BSME 1
Student Registration
Student Number *
e.g. 2023300845
First Name *
e.g. Juan
Last Name *
e.g. Dela Cruz
Course *
— Select Course —
Year Level *
— Select Year —
+ Register Student
Reset

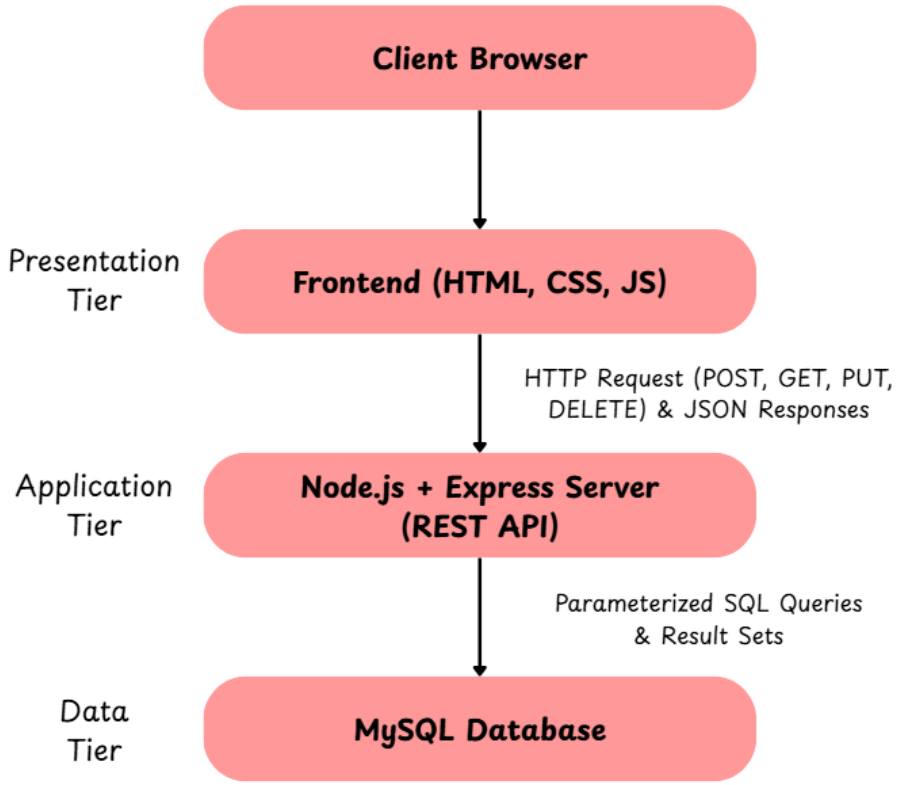

Student Records 11
Search by name, number, or course...
All Courses
Refresh
ST. NO. FULL NAME COURSE ACTIONS
2023300007 Aquino, Elena BSEd Edit Del
2023300005 Bautista, Luz BSEE Edit Del
2023300009 Castillo, Sofia BSCS Edit Del
2023300003 Cruz, Ana BSCE Edit Del
2023300004 Garcia, Pedro BSIT Edit Del
2023300010 Mendoza, Roberto BSCE Edit Del


2023300003 Cruz, Ana BSCE Edit Del
2023300004 Garcia, Pedro BSIT Edit Del
2023300010 Mendoza, Roberto BSCE Edit Del
2023300599 Pongasi, DBFlowers BSChem Edit Del
2023300008 Ramirez, Carlos BSIT Edit Del
2023300002 Reyes, Jose BSCS Edit Del
2023300001 Santos, Maria BSIT Edit Del
Showing 1-10 of 11 students
Prev
Page 1 of 2
Next
© 2026 Student Record Management System - IT323 - Midterm PIT
Developed by Bene Gabriel Rosel - Joeremy Christie Ong - Vince Roy Lim
Built with Node.js · Express · MySQL · Vanilla JS

Technical Report

• System Architecture

The Student Record Management System (SRMS) is a full-stack web application built following **the three-tier architecture** model



Project Structure

- server.js - Entry point, mounts middleware + routes
- package.json - Dependencies and npm scripts
- .env.example - Environment variable template (copy to .env)
- config/db.js - MySQL connection pool (mysql2/promise + dotenv)
- routes/students.js - All /api/students route handlers
- public/
  index.html - Single-page frontend
  style.css - Full design system (Grid + Flex)
  script.js - All Vanilla JS logic
- student_management.sql - Database schema + seed data

- **Database Design Rationale**

I. Schema Design

Column	Data Type	Constraints	Description
id	INT	AUTO_INCREMENT PRIMARY KEY	Unique row identifier
student_number	VARCHAR(20)	UNIQUE NOT NULL	Official student ID number
first_name	VARCHAR(100)	NOT NULL	Student's first name
last_name	VARCHAR(100)	NOT NULL	Student's last name
course	VARCHAR(100)	NOT NULL	Enrolled Academic Program
year_level	INT	NOT NULL CHECK (year_level BETWEEN 1 AND 5),	Current year level (1-4)
created_at	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Record creation timestamp

- The AUTO_INCREMENT primary key ensures each record has a unique, system-generated identifier that is safe to use in API routes.
- The UNIQUE constraint on student_number enforces business rules at the database level, preventing duplicate enrollment entries even if application- level validation is bypassed.

II. SQL Operation implemented

Operation	SQL statement used
Insert student	INSERT INTO students (student_number, ...) VALUES (?, ?, ...)
Select all student (ordered ASC)	SELECT * FROM students ORDER BY last_name ASC
Select student by ID	SELECT * FROM students WHERE id = ?
Update student	UPDATE students SET student_number=?, first_name=?, last_name=?, course=?, year_level=? WHERE id=?
Delete student	DELETE FROM students WHERE id = ?
Search (using LIKE)	SELECT * FROM students WHERE first_name LIKE ? OR last_name LIKE ? OR student_number LIKE ? OR course LIKE ?

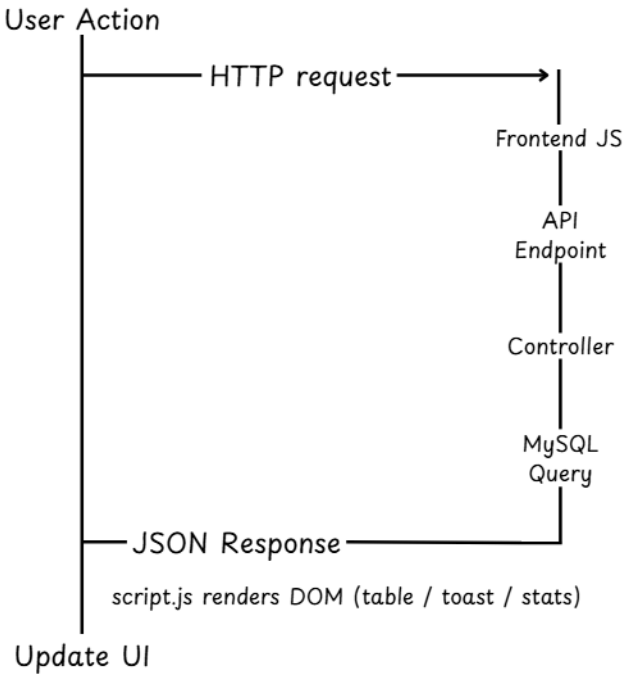
III. Normalization

1NF: All columns are atomic; no repeating groups

2NF: No partial dependencies (single-column PK)

3NF: No transitive dependencies; `course` is a free-text attribute intentionally not normalized into a separate table at this stage

- **API Flow Explanation**



- Error Handling Strategy
  400 - Missing required fields or duplicate `student_number` (caught via `ER_DUP_ENTRY` MySQL error code)
  404 - Row lookup before update/delete; returns early with message
  500 - Unexpected DB errors; caught in `try/catch`, logged server-side, generic message returned to client

- **Challenges Encountered**

- Problem: Express matched `/api/students/search/:keyword` as `/api/students/:id` (interpreting `"search"` as an ID), triggering a MySQL type mismatch error.
  Solution: The `search/:keyword` route was declared **above** the `/:id` route in `routes/students.js`. Express evaluates routes in registration order, so placing the specific literal path first ensures it matches before the generic wildcard.
- Problem: Firing an API call on every keystroke caused excessive server requests and visual flickering.
  Solution: Implemented a 400 ms debounce timer in `script.js` using `clearTimeout` / `setTimeout`. The search API is only called once the user pauses typing, reducing network calls by ~80% during normal use.
- Problem: Rendering student data directly via `innerHTML` with unsanitized data could allow stored XSS if malicious content was inserted into the database.

Solution**:** An `escHtml()` utility function was implemented to escape `&`, `<`, `>`, `"`, and `'` characters before any user-provided data is inserted into the DOM via `innerHTML`.

- Problem: At viewport widths ≤768px, the navigation links overlapped with the brand and badge elements, breaking the layout.
  Solution: A hamburger button (`#nav-toggle`) was added to the navbar. On mobile, the nav links are hidden by default and revealed as a full-width vertical dropdown when the hamburger is clicked. The button animates into a close (✕) icon when the menu is open using CSS-only bar transforms.

• Future Improvements

- User Authentication
  Implement JWT-based authentication with role-based access control (admin vs. registrar) using `jsonwebtoken` and `bcrypt` for password hashing.
- Pagination
  Add server-side pagination (`LIMIT / OFFSET`) for scalability when the student population grows beyond a few hundred records. The API would accept `?page=1&limit=20` query parameters.
- Normalized Courses Table
  Create a separate `courses` master table with a foreign key from `students.course_id`, enabling course management (add/archive courses) without modifying student records directly
- Data Export
  Allow administrators to export the student list as a CSV or PDF report via a `GET /api/students/export` endpoint using `json2csv` or `pdfmake`.
- Automated Testing
  Add unit tests for route handlers using `Jest` + `supertest`, and integration tests against a dedicated test database to ensure regression safety during development.

Members:

Ong, Joeremy Christie

Rosel, Bene Gabriel

Lim, Vince Roy